

Claim Consistency Tracker — Hackathon Idea Report

Built on: cognee (AI memory / knowledge graph layer) **Team size:** 2 **Timeline:** 4 days **One-line pitch:** A system that holds a politician accountable to their own record — extracting claims from a live or recent speech and checking them against a temporal knowledge graph of everything they've said before, surfacing contradictions and numeric drift with full sourcing.

1. The reframe (read this first)

The original framing — "detect when a politician lies, exaggerates, or distorts" — is not buildable in 4 days, and isn't really buildable at all, because *lying* requires knowing ground truth and intent, not just text. What's actually checkable is **self-contradiction**: did this person say something today that conflicts with something they said before, on the record, with a date attached?

This project is now scoped as a **claim consistency tracker**, not a lie detector. Same demo energy, dramatically more defensible, and it maps directly onto what cognee is actually good at.

2. Why cognee specifically (not just "a vector DB")

A naive version of this dumps speech transcripts into embeddings and does similarity search. That's a weak use of cognee and won't impress anyone who knows the tool. The version worth building leans on three things cognee does that generic RAG doesn't:

- **Triplet-based graph extraction** — cognee turns text into subject-relation-object nodes, so "Politician X claimed unemployment is 4%" becomes a structured, queryable fact rather than a text blob.
- `temporal_cognify` — a first-class temporal pipeline that builds an event-and-timestamp graph at ingestion time.
- `SearchType.TEMPORAL` — native before/after/between query support, so "what did they say about this topic before today" is a graph traversal, not a hack.
- **Node Sets / ontologies** — let you scope retrieval per politician and per topic, and encode a topic taxonomy (economy → inflation/unemployment/GDP) so claims cluster sensibly.

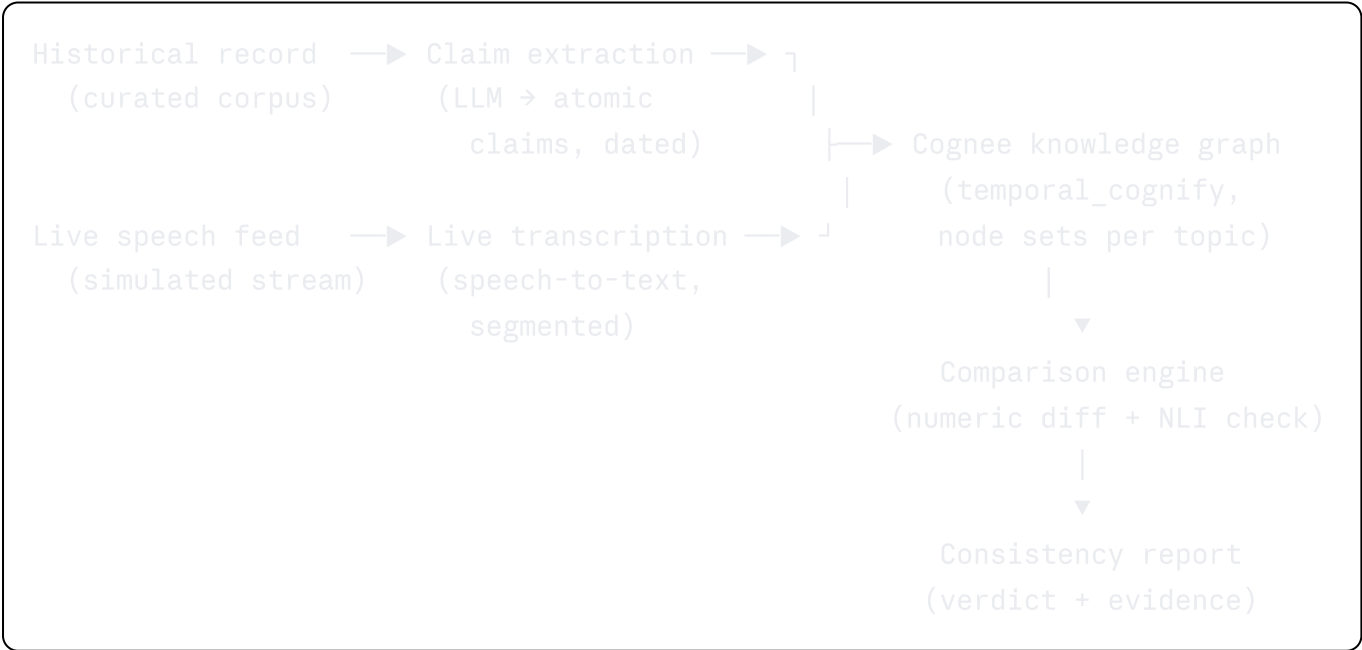
The pitch to judges: *we built the comparison logic ourselves and used cognee's temporal graph as the actual memory substrate* — not "we called an LLM and called it memory."

3. MVP scope — what's in, what's cut

In scope	Cut for this hackathon
1 politician, 2-3 topics	Multiple politicians, full topic coverage
Hand-curated historical corpus (50-150 statements)	Live web scraping pipeline
English-language statements only	Multilingual (Hindi/Tamil/regional) claim matching
Simulated live feed (pre-transcribed speech, streamed sentence-by-sentence)	True live audio ingestion
Numeric diff + LLM contradiction check	Full NLI model training, sarcasm/rhetoric detection
Self-consistency only (vs. their own past statements)	Fact-checking against objective/external ground truth

This scope cut is the single most important decision in this report. Multilingual support and live scraping are each their own multi-week project — cutting them isn't lowering ambition, it's what makes the rest achievable.

4. Architecture (pipeline)



Comparison engine — own this part, don't outsource it to an LLM black box:

- Deterministic path for anything numeric (percentages, dates, counts) — compute drift directly, explainable line-by-line.

- LLM contradiction/entailment classifier only for qualitative claims (does claim B contradict, support, or merely extend claim A) — narrow job, not the final verdict-writer.

Output labels — avoid binary "true/false" or "lie" framing:

- *Consistent with prior statements*
- *Contradicts statement from [date, source]*
- *No prior record on this topic*
- *Stated figure differs from earlier figure by X%*

Always show the matched historical claim with date + source link next to the verdict. The evidence is the product.

5. Day-by-day build plan (2 people, 4 days)

Day	Person A	Person B	End-of-day target
1 — Foundation	Stand up cognee locally/cloud; get <code>temporal_cognify</code> working on a toy dataset; design the graph schema (politician / topic / claim / date)	Source + clean 50-150 historical statements for one politician (news archives, official transcripts, parliamentary records); write the claim-extraction prompt	Seed corpus ingested into cognee as dated, topic-tagged claims
2 — Core logic	Build temporal retrieval: given a new claim, fetch top-k relevant prior claims "before now" via <code>SearchType.TEMPORAL</code>	Build the comparison engine: numeric diff logic + LLM contradiction classifier; define the verdict schema	End-to-end run on 5-10 hand-picked test claims, with sane verdicts
3 — Live + UI	Build the live-feed simulator: stream one real (pre-transcribed) speech sentence-by-sentence with a short delay; wire claim extraction + comparison to it live	Build the dashboard: live transcript pane + flagged-claim cards showing verdict, evidence, source link	Full pipeline working end-to-end, ugly but functional

Day	Person A	Person B	End-of-day target
4 — Polish + demo	Bug fixes, edge cases (no prior record, ambiguous claims), stress-test the demo path 5+ times	Pitch narrative, slides/this report, UI polish	A reliable 3-5 minute demo with 2-3 pre-verified standout examples

Important practical tip: LLM calls are nondeterministic. Pre-run and cache the results for your actual demo claims well in advance — don't rely on live API calls being well-behaved in front of judges.

6. Feasibility assessment

Score: 7/10 for the scoped MVP above. Drops to ~4/10 if you attempt scraping, multiple politicians, or multilingual matching in the same 4 days.

Risk	Likelihood	Mitigation
cognee learning curve eats day 1	Medium	Do a 1-2 hour "hello world" ingest+query before committing to your schema
Historical corpus is messier/sparser than expected	High	Hand-curate a small, clean corpus rather than scraping; quality over coverage
Claim extraction produces noisy/wrong atomic claims	Medium-High	Budget real iteration time on the prompt on day 1-2; test on 10+ examples before trusting it
Comparison engine flags false positives (sarcasm, hypotheticals, rhetorical claims)	High	Expected and unavoidable — frame this honestly in your pitch rather than hiding it
Live demo breaks under judge scrutiny	Medium	Cache/pre-verify your specific demo examples; never demo cold
Scope creep (multilingual, more politicians, real scraping)	High if undisciplined	The cut list in Section 3 exists for exactly this reason — hold the line

7. Suggested tech stack

- **Memory layer:** cognee (Python), local graph + vector backend
 - **Claim extraction & comparison:** any capable LLM API, structured/JSON output
 - **Transcription (pre-processed, not live):** Whisper or AssemblyAI, run once ahead of the demo
 - **Backend:** FastAPI or similar, simple WebSocket for the simulated live stream
 - **Frontend:** lightweight dashboard (React or even a simple HTML/JS page) — transcript pane + claim cards
-

8. Demo script (suggested)

1. 30 seconds: the reframe — "we're not claiming to detect lies, we're tracking consistency against the public record."
 2. Show the graph being built from historical statements (visual proof cognee's temporal layer is real, not decorative).
 3. Stream the live/simulated speech. Let 1-2 genuinely consistent claims pass through quietly.
 4. Hit your pre-verified standout example: a claim that contradicts an earlier dated statement, with the evidence card showing both side by side.
 5. Show a numeric-drift example (a statistic that changed) — this is your most defensible, hardest-to-dispute case.
 6. Close with the honest caveat slide: what this can't do yet (sarcasm, rhetoric, external fact-checking) — this reads as maturity, not weakness, to judges.
-

9. Value & competitive landscape

This genre already has real-world precedent, which is good news — it means the problem is validated, not invented:

- **PolitiFact's "Flip-O-Meter"** has manually tracked U.S. politicians' position changes for years.
- **Rappler** (a major Philippine news organization) built an ontology-backed politics knowledge graph specifically to track politicians and surface disinformation patterns over time.
- Broader "AI for politics" tooling in 2026 leans toward legislative tracking and sentiment/opinion clustering (e.g. bill-comparison tools, deliberation platforms) — not claim-level personal consistency tracking.

No dedicated AI-native, real-time, graph-based self-consistency product for politicians surfaced in a search of the current landscape. That's a genuine, underserved niche — which is also exactly why a hackathon-scale prototype of it is a credible, novel entry rather than a reskin of something that already exists.

The honest limitation: a productionized version naming real politicians would need real editorial and legal review before any public deployment. For a hackathon demo this is a non-issue, but it's worth naming in your pitch as "future work," since it shows awareness rather than naivety.

10. Resume-worthiness — the honest version

Strong case for: general AI-engineering, applied NLP, and full-stack roles, especially at large product companies. It demonstrates structured LLM output design, knowledge-graph reasoning, an end-to-end shipped system under real time pressure, and a use case a recruiter understands in 10 seconds — which matters more for resume impact than people expect.

One thing to flag honestly: this is, structurally, an LLM-orchestration / multi-agent-reasoning project — the same shape as a project you've already built (the multi-agent debate system). It's a genuinely different *domain* and adds the cognee/knowledge-graph layer plus real-time retrieval, which is real new surface area — but on a resume, two projects with a similar silhouette (LLM agents debating/comparing claims) read less diversely than one of those plus something in a completely different lane. Worth deciding *before* the hackathon whether this is "the AI-systems project that supersedes the older one" or "an addition" — and writing the resume bullet to emphasize what's distinct (temporal graph retrieval, real-time pipeline) rather than what's similar.

Less directly relevant to: a GSoC systems/performance narrative (libvips, OpenCV, LLVM, FFmpeg-style orgs). This project won't strengthen a "I write fast, low-level code" story the way a C++ stacking pipeline does. It's a complement that shows breadth, not a substitute for the systems-track portfolio piece.

11. Stretch goals (only if ahead of schedule)

- A second politician for cross-comparison ("who's more consistent on topic X")
- A simple confidence score on each verdict, surfaced in the UI
- Multi-hop graph traversal: connecting claims through related entities (e.g. a claim about tax cuts linked to an earlier claim about growth) rather than only same-topic matches